NATURAL LANGUAGE PROCESSING

Automated Financial Customer Complaint Classification

Team Project Report

Programme	SAII BBA
Semester	3
Academic Year	2026-27
Submission Date	9 October 2026

Team Members

Name	PRN
Shorya Jaiswal	25030422113
Namir Al Balushi	25030422151
Abdullah Al Hinai	25030422149
Yousuf Altom	25030421050

Faculty Guide

Nabanita Choudhury, Assistant Professor

SYMBIOSIS ARTIFICIAL INTELLIGENCE INSTITUTE (SAII)

Constituent of Symbiosis International (Deemed University)
Lavale, Pune - 412115

Contents

1. Abstract 3

2. Introduction 3

3. Problem Statement and Objectives 4

4. Dataset Description 4

5. Data Preprocessing 6

6. Feature Representation / Embeddings 7

7. Proposed Methodology / System Architecture 8

8. NLP Model / Technique Used 9

9. Implementation 10

10. Results and Evaluation 12

11. Error Analysis / NLP Failure Analysis 17

12. Ethical Considerations and Limitations 18

13. Conclusion 19

14. Future Scope 19

15. References 20

16. GitHub Repository 21

17. Team Contribution 21

18. Annexure (if required) 22

This report covers how we prepared the data, trained the models, evaluated them, audited the results and built the demo for the complaint classifier. All figures and tables come from the saved outputs and test predictions of our own run.

1. Abstract

When a customer complains to a bank, someone has to read the text and send it to the right product team. We built a supervised NLP pipeline that does this first step automatically. It takes complaints from the Consumer Financial Protection Bureau (CFPB) Consumer Complaint Database and sorts them into nine product categories. We worked from a fixed snapshot of 10,000 real records. After cleaning, removing conflicting labels and duplicates, and dropping very small categories, 5,042 usable narratives were left. We turned the text into TF-IDF word unigrams and bigrams and compared logistic regression, a linear support vector machine (SVM) and a random forest against a majority-class baseline. The linear SVM with C=1 had the best validation Macro-F1, so we picked it before looking at the test set. On 757 held-out test complaints it got 673 right, which is 88.90% accuracy, with a Weighted-F1 of 0.8842 and a Macro-F1 of 0.6834. Our audit showed that short complaints were classified less accurately than long ones, and that all six payday, title or personal-loan test cases went wrong (F1 of zero). We could not draw a firm conclusion on formal versus informal writing, because only five complaints matched our informal-short rule. We also explain individual predictions using exact feature contributions, and we built a browser demo and a one-page model card. The classifier is meant to help a human decide where a complaint goes, not to replace that decision.

2. Introduction

Banks, financial companies and regulators get complaints about credit reports, debt collection, mortgages, cards and many other products. One complaint can mention several problems, personal stories and a lot of overlapping financial terms. Reading each one by hand is slow, and different people may categorise the same complaint differently. Our project only deals with the first step, which is routing a complaint to a product category. It does not decide whether the complaint is valid or whether compensation is owed.

NLP lets us turn these narratives into numbers that a classifier can learn from. Here the product field is the label and the complaint text is the input. The model learns which words and short phrases tend to go with which product. We then test it on complaints it has not seen, taken from the same snapshot, to see how well that learning carries over.

We also wanted to look beyond a single accuracy number. Longer complaints probably contain more clues about the product, while short or casual ones may not. Class imbalance can also hide poor results on smaller categories. So alongside overall accuracy we report per-category scores, comparisons by text length, real examples of failures and word-level explanations.

3. Problem Statement and Objectives

The task is to assign a consumer financial complaint to one of the product categories in our dataset. The input is free text. The output is one predicted category, along with the model score and the words that pushed the prediction, so that a human reviewer can check it.

- Build a pipeline that runs end to end on real CFPB complaint narratives and product labels.
- Clean the noisy text, standardise product names, remove duplicates and make sure the same cleaned text never ends up in more than one data split.
- Compare logistic regression, linear SVM and random forest on TF-IDF features using validation data.
- Evaluate the chosen model with accuracy, precision, recall, Macro-F1, Weighted-F1 and a confusion matrix.
- Check performance by complaint length and by simple writing-style rules, and be clear about what those checks can and cannot show.
- Look at the model's most important features overall, and at the exact word contributions behind single predictions.
- Provide a Colab notebook that can be run, a live browser demo and a one-page model card.

4. Dataset Description

We used the CFPB Consumer Complaint Database, a public collection of complaints about consumer financial products and services [1, 2]. We did not use the whole current database. Instead we saved a 10,000-record snapshot from the API, covering complaints received between 26 August and 4 September 2024. The notebook carries a compressed copy of the CSV and checks its SHA-256 hash before using it, so the demo does not break if the API output changes.

Table 1. Fields used from the source snapshot

Field	Use in this project
Consumer complaint narrative	Text input for cleaning, feature extraction and classification.
Product	Source label, mapped to the project categories.
Complaint ID	Source record identifier retained for traceability.
Date received	Describes the snapshot period; not supplied as a predictive feature.

The cleaning step keeps only English alphabetic text. We did not add language detection or test on other languages, so we cannot say how the model performs on non-English or mixed-language complaints. Also, the product label is just the category recorded in the source data. It is not a verified answer about every issue mentioned in the complaint.

4. Dataset Description

Class distribution and scope

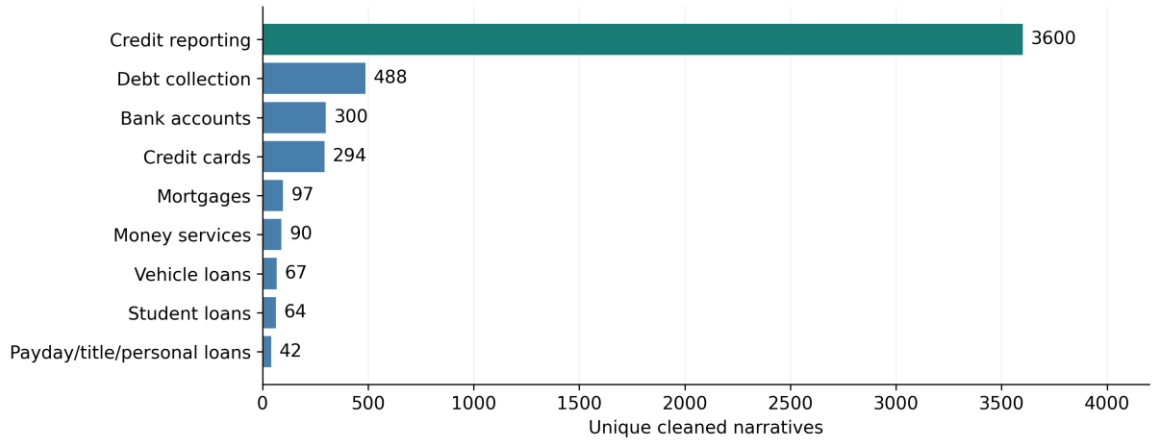


Figure 1. Distribution of the 5,042 unique cleaned narratives. Credit reporting accounts for 71.40% of the retained data.

Table 2. Cleaned dataset and held-out test counts by category

Category	Clean count	Test count
Credit reporting	3600	540
Debt collection	488	74
Bank accounts	300	45
Credit cards	294	44
Mortgages	97	15
Money services	90	14
Vehicle loans	67	10
Student loans	64	9
Payday/title/personal loans	42	6

Credit reporting makes up 3,600 of the 5,042 cleaned narratives and 540 of the 757 test records. Because of this, a model that always says “credit reporting” already looks fairly accurate, even though it is useless for every other category. That is why we used Macro-F1, which counts each category equally, as our main metric for choosing a model.

We dropped two categories because they had fewer than 40 unique narratives after filtering: Prepaid cards (22) and Debt/credit management (15). The final model only predicts the nine remaining categories and has never seen those two products.

5. Data Preprocessing

The same cleaning function is applied to the training data and to any new complaint. It decodes HTML entities, converts text to lowercase, removes URLs, email patterns, numbers and redaction markers such as XXXX, and replaces any character that is not an English letter, space or apostrophe with a space. Extra spaces are then collapsed. This also strips hashtag and mention symbols but keeps the words next to them. Emojis are removed too, so any emotion they carry is not modelled.

Narratives with fewer than three cleaned words are rejected. Source product names are mapped to our categories through a lookup dictionary. If identical cleaned text appears with different labels, we remove every record in that group. The remaining identical narratives are deduplicated before the split. We did not remove stop words or apply stemming or lemmatisation. Keeping words like "not" preserves denial language, and bigrams keep a bit of local word order.

Table 3. Measured sequential filtering of the actual snapshot

Stage	Remaining	Removed
API snapshot	10000	0
Nonempty required fields	10000	0
At least 3 cleaned words	9993	7
Conflicting labels excluded	9803	190
Normalised duplicates removed	5079	4724
Classes with at least 40 records	5042	37

Every one of the 10,000 records in our snapshot already had the required fields filled in, so the missing-field check removed nothing. Overall, 4,958 records were removed. Of these, 4,724 were repeated narratives after normalising, 190 had conflicting labels, 7 were too short after cleaning and 37 belonged to rare categories. Most of the drop comes simply from deduplication.

Example of text cleaning

Original: "My MORTGAGE payment was wrong!!! Visit https://example.com, account XXXX 12345."

Cleaned: "my mortgage payment was wrong visit account"

TF-IDF does its own tokenisation after cleaning, and it only keeps tokens of two or more characters. Because of this, the word count we use for eligibility and the number of feature tokens can differ slightly. Our cleaning also removes dates and amounts, which could be useful in other financial NLP tasks.

6. Feature Representation / Embeddings

We represent each complaint as a sparse vector using term frequency-inverse document frequency (TF-IDF). Term frequency counts how often a word or phrase appears in a complaint. Inverse document frequency lowers the weight of terms that show up in many complaints. The result is that words which help tell products apart, like “mortgage”, “credit card” or “debt collection”, count for more [3].

We include single words (unigrams) and pairs of neighbouring words (bigrams). This lets the model tell “credit report” apart from the word “credit” on its own. The vocabulary and IDF values are learned from the training split only. Validation, test and live inputs are transformed using those fixed values.

Table 4. TF-IDF settings used in the executed experiment

Setting	Value	Reason
ngram_range	(1, 2)	Capture words and short phrases.
min_df	2	Exclude terms appearing in fewer than two training narratives.
max_df	0.98	Exclude terms appearing in more than 98% of training narratives.
max_features	30,000	Limit the feature space and resource use.
sublinear_tf	True	Use one plus the logarithm of the term count.
norm	l2	Normalise each complaint vector to unit length.
use_idf / smooth_idf	True / True	Apply smoothed inverse document frequency.
stop_words	None	Retain terms such as negations.

The training matrix has 3,529 rows and 30,000 columns. Most entries are zero, because a single complaint only uses a small part of the vocabulary. TF-IDF works well as a small, explainable baseline and is fast with linear models. We did not train Word2Vec, use pretrained embeddings or fine-tune a transformer. Those are possible next steps, not work we have done.

Small illustrative example

Take a complaint about a wrongly charged mortgage payment. The words “mortgage” and “servicer” switch on features, and the SVM combines their values with its category weights. A word that is not in the learned vocabulary adds nothing, which is one reason odd spellings or unseen product names can cause mistakes.

7. Proposed Methodology / System Architecture

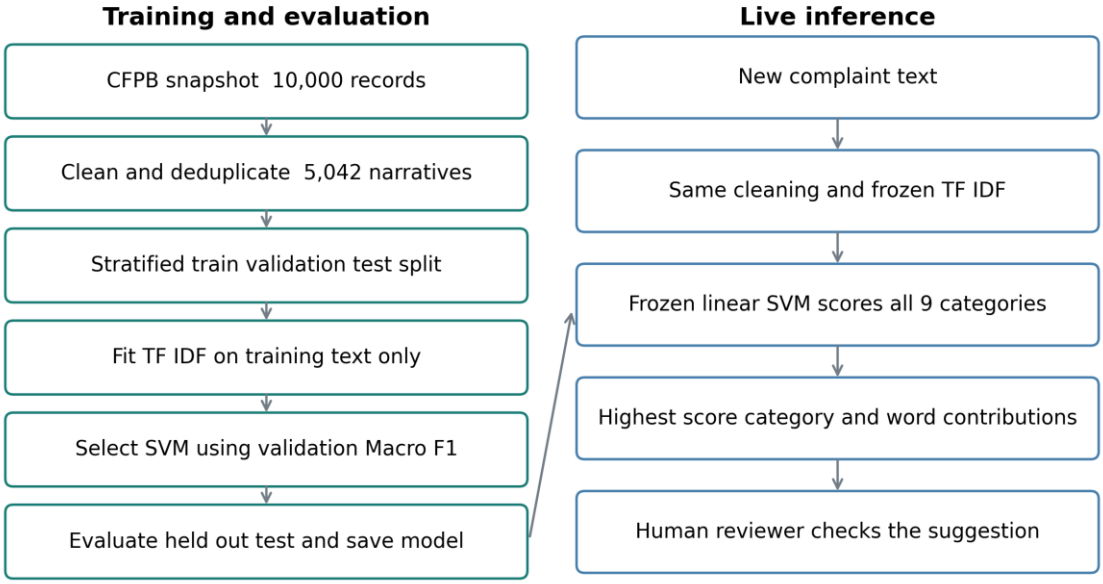


Figure 2. Training and live-inference workflows. The deployed classifier uses the frozen vocabulary and model weights.

Table 5. Stratified data splits with random seed 42

Split	Narratives	Purpose
Training	3,529	Fit the TF-IDF vocabulary, IDF values and candidate classifiers.
Validation	756	Choose the model and tested regularisation setting.
Test	757	Evaluate the chosen model and conduct the audit.

The 70/15/15 split is done with two stratified random splits, so the exact counts differ by a few records because of rounding. Each split keeps roughly the same product mix. We save hashes of the cleaned narratives and use assertions to check that no identical text appears in more than one split.

The chosen model is trained on the training split only and is never refitted on the test set. We only touched the test set after the model had been selected. Deduplication, label grouping and the minimum class size rule were all decided before splitting. This is a fixed-snapshot evaluation, not a forecast over time, and similar templates with different wording can still sit in different splits.

8. NLP Model / Technique Used

We compared three kinds of supervised classifier against a majority-class baseline. Logistic regression learns a linear link between features and category scores. A linear SVM finds separating boundaries and suits sparse text features. Random forest combines many decision trees and gives us a nonlinear option [4-6]. All of them use the same TF-IDF features and the same training split.

Table 6. Candidate classifiers and key settings

Candidate	Settings
Majority baseline	DummyClassifier with strategy most_frequent.
Logistic Regression C=1	C=1; balanced class weights; max_iter=1,500; random_state=42.
Logistic Regression C=4	C=4; balanced class weights; max_iter=1,500; random_state=42.
Linear SVM C=0.5	LinearSVC; C=0.5; balanced class weights; random_state=42.
Linear SVM C=1	LinearSVC; C=1; balanced class weights; random_state=42.
Random Forest	150 trees; maximum depth 40; minimum leaf size 2; balanced_subsample; n_jobs=-1; random_state=42.

Balanced class weights make mistakes on rare categories cost more during training. This helps with imbalance, but it does not create new evidence for rare products. C controls regularisation in the linear models. We only tried a few values, so this is a small comparison and not a full hyperparameter search.

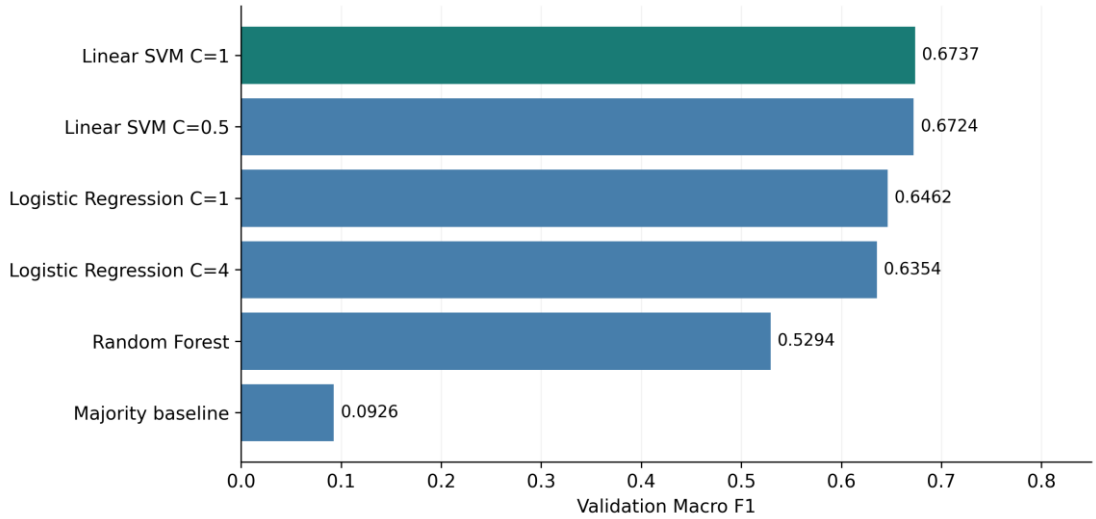


Figure 3. Validation Macro-F1 picks Linear SVM C=1. The test set was not used to choose the winner.

The selected SVM got a validation Macro-F1 of 0.6737, only slightly above 0.6724 for C=0.5. The gap is tiny, so we would not claim that C=1 is the best value in general. The vectoriser and SVM are saved together as one joblib pipeline.

9. Implementation

All the code is in one Jupyter notebook that runs on Google Colab. Its cells load the saved dataset, check the source hash, prepare the labels and text, create the splits, train the candidates, pick the SVM, evaluate it, run the audit and generate explanations. It also has a text-entry widget and a direct prediction cell for demonstrating in class.

Table 7. Major implementation components

Component	Role
Python and standard library	Text cleaning, JSON manifests, hashing and compressed dataset decoding.
pandas and NumPy	Read CSV records, organise labels and metrics, and perform numerical operations.
scikit-learn	TF-IDF, data splitting, classifier training and evaluation [3-7].
Matplotlib	Performance charts, confusion matrix and contribution plots [9].
joblib	Save and load the fitted feature and classifier pipeline.
ipywidgets	Interactive text-entry demo in a compatible notebook runtime.
HTML CSS and JavaScript	Run the exported frozen classifier and display outputs in a browser.

Recorded notebook evaluation output

```
Correct: 673 / 757
Test accuracy: 88.90%
Macro-F1: 0.6834 | Weighted-F1: 0.8842
95% bootstrap accuracy interval: [0.8666, 0.9102]
```

Figure 4. Evaluation output from the executed notebook, shown as saved text.

The notebook keeps its executed output cells and writes files such as test_metrics.json, classification_report.csv, test_predictions.csv and audit.csv. The numbers in this report come from these files. We also recomputed the metrics from the 757 saved test predictions and they matched the recorded results.

9. Implementation

Demonstration and runtime

Recorded notebook prediction output

```
Input: My mortgage servicer applied my payment incorrectly and charged a late fee.
Predicted category: Mortgages
Use: suggestion for a human reviewer.
Raw decision score: 0.627661 (not a probability)
Intercept -1.010147 + contributions 1.637808 = 0.627661
```

Figure 5. Recorded output of the mortgage demo, with the raw SVM score and its exact breakdown.

This example is an input we wrote for the demo, so it is not one of the held-out test complaints. The model predicts Mortgages with a raw decision score of 0.627661. The notebook prints that this is a suggestion for a human reviewer and that the score is not a probability.

Browser demonstration

The website has a live complaint-entry box, example inputs, the predicted product, the most influential terms, a performance tab, real error cases and a downloadable model card. We export the IDF values, vocabulary, class coefficients and intercepts from the saved Python pipeline. JavaScript then does the same cleaning, unigram and bigram extraction, sublinear weighting, L2 normalisation and linear scoring.

To check this, we compared the browser output with Python on all 757 held-out complaints plus four demo inputs. All 761 predicted labels matched, and the largest difference in decision score was below 0.000000000000008. The contribution sums also reproduced the model scores. This only confirms that the browser version matches the trained model. It does not make the model more accurate.

Hosted demonstration

The hosted site needs account access. The standalone file Financial_Complaint_Live_Demo.html has the same model, outputs and model card and works offline in a browser. Text typed into the demo is processed locally and is not sent to any prediction server.

Verified execution environment

The recorded run used Python 3.12.14, NumPy 2.3.5, pandas 2.2.3, scikit-learn 1.8.0 and joblib 1.5.3. These are the versions from our verification run, and you do not need to upgrade Colab's NumPy to match them. The corrected notebook checks the libraries Colab already has instead of reinstalling its scientific stack. Starting from a fresh runtime avoids leftover packages from an earlier failed installation.

10. Results and Evaluation

Overall held-out performance

Table 8. Selected model and majority baseline on the same 757 test complaints

Model	Accuracy	Macro-F1	Weighted-F1
Majority baseline	71.33%	0.0925	0.5940
Linear SVM C=1	88.90%	0.6834	0.8842

The selected model classifies 673 complaints correctly and 84 wrongly. Its accuracy is 17.57 percentage points above the majority baseline. The 95% percentile bootstrap interval for accuracy is 86.66% to 91.02%, from 1,000 resamples of the test results with seed 42. This interval only reflects sampling variation in this particular test set. It does not cover changes in complaint wording, product mix or future data.

Accuracy is the share of complaints put in the right category. Precision tells us how many of the predictions for a category were correct, and recall tells us how many of the complaints in that category were found. F1 balances the two. Macro-F1 averages the F1 of each category equally, while Weighted-F1 weights each category by its test support [7]. We set zero_division=0 when precision or F1 is undefined for a category.

Why Weighted-F1 is higher than Macro-F1

Credit reporting is 540 of the 757 test complaints (71.33%) and has an F1 of 0.9460, so it pulls the weighted average up. Payday/title/personal loans has only six test complaints and an F1 of 0.0000. That barely moves Weighted-F1, but in Macro-F1 it counts as a whole category. The 0.2009 gap between the two scores shows how uneven the performance is across products.

Table 9. Validation comparison used before test evaluation

Candidate	Accuracy	Macro-F1	Weighted-F1
Linear SVM C=1	88.23%	0.6737	0.8769
Linear SVM C=0.5	88.23%	0.6724	0.8783
Logistic Regression C=1	84.79%	0.6462	0.8523
Logistic Regression C=4	86.38%	0.6354	0.8637
Random Forest	85.05%	0.5294	0.8377
Majority baseline	71.43%	0.0926	0.5952

We kept validation and test results separate. Only the selected SVM and the baseline were evaluated on the test set, so the held-out data was never used for a second round of model selection.

10. Results and Evaluation

Performance across product categories

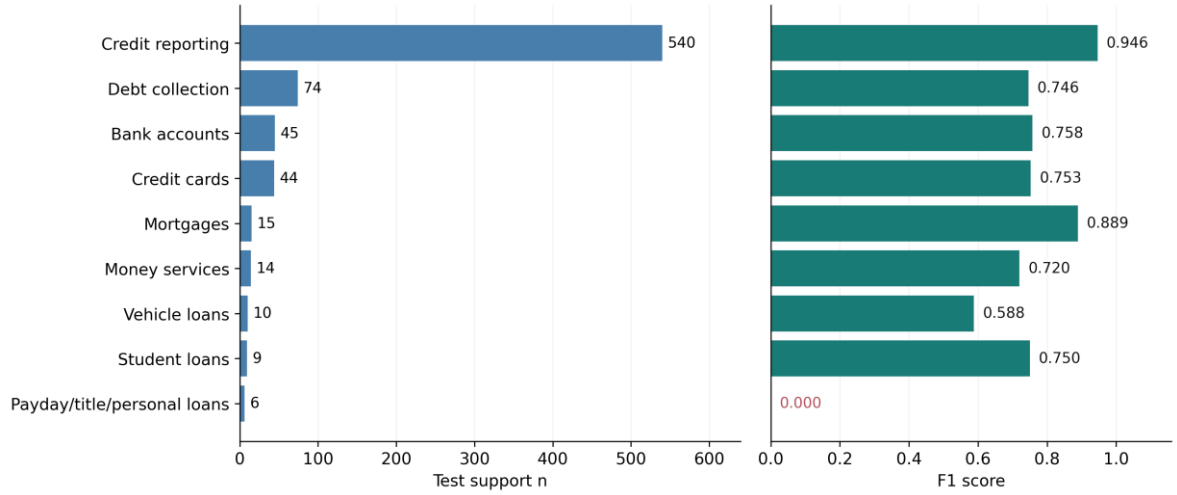


Figure 6. Category support and F1 side by side. The largest category does well, while the six payday/title/personal-loan cases have zero F1.

Table 10. Per-category held-out metrics

Category	Precision	Recall	F1	n
Bank accounts	0.720	0.800	0.758	45
Credit cards	0.714	0.795	0.753	44
Credit reporting	0.935	0.957	0.946	540
Debt collection	0.779	0.716	0.746	74
Money services	0.818	0.643	0.720	14
Mortgages	1.000	0.800	0.889	15
Payday/title/personal loans	0.000	0.000	0.000	6
Student loans	0.857	0.667	0.750	9
Vehicle loans	0.714	0.500	0.588	10

Mortgages has an F1 of 0.8889, but only 15 test complaints, so a few more mistakes would change that score a lot. Vehicle loans has a recall of 0.5000, meaning the model found only five of its ten test complaints. Payday/title/personal loans needs more data and a closer look. Overall, we cannot say the model is equally reliable for all nine products.

10. Results and Evaluation

Confusion matrix

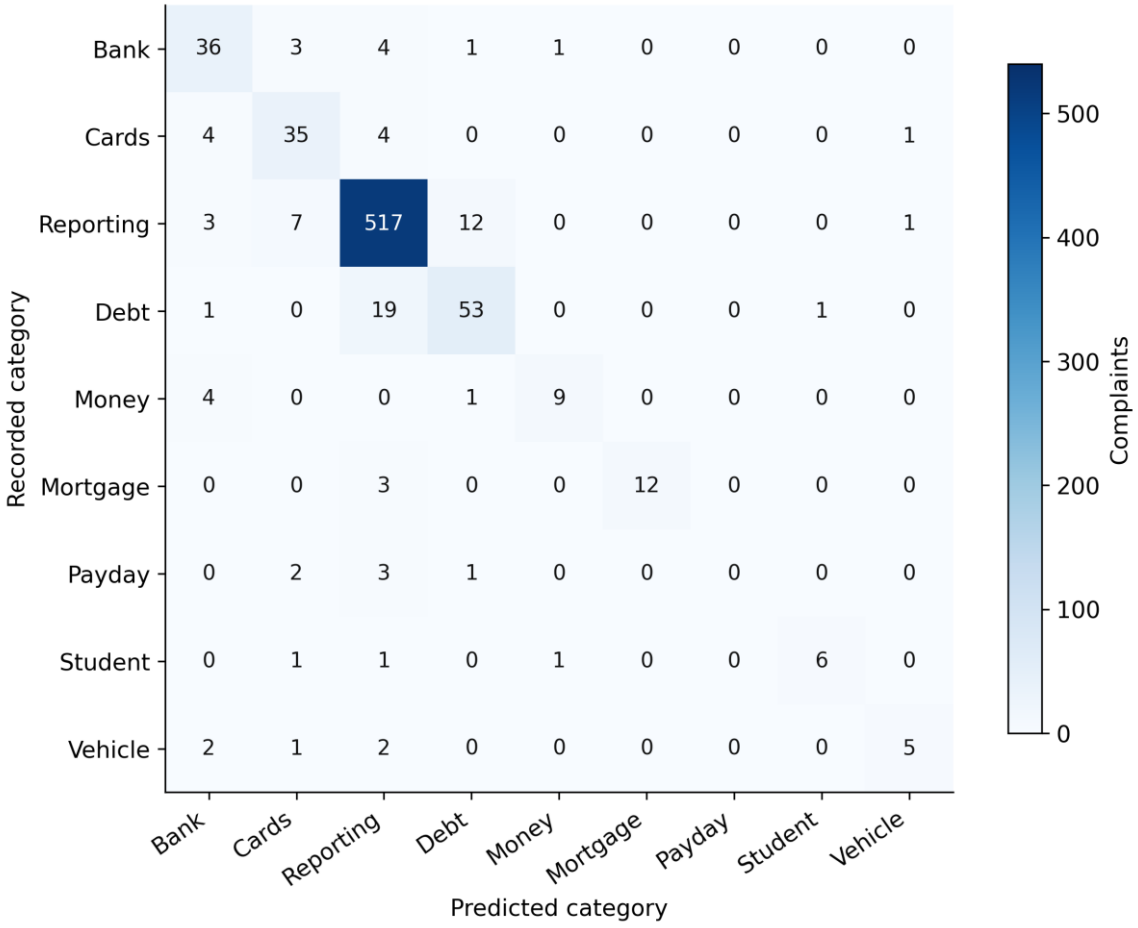


Figure 7. Counts of recorded versus predicted product categories. Diagonal cells are correct; all other cells are errors.

The biggest confusion is between debt collection and credit reporting. Nineteen debt-collection complaints were predicted as credit reporting, and twelve credit-reporting complaints were predicted as debt collection. Many of these complaints are about a debt showing up on a credit report, so the wording fits both products. We still treat the recorded product label as the correct answer.

Credit-card and bank-account language overlaps too. Four credit-card complaints were classified as bank accounts. Words like “account”, “access” or “fraud” do not always point to one product. A model based on local word patterns can miss which product the complaint is really about.

The figure shows raw counts, not row percentages, so the real test support stays visible. Because of that, the large credit-reporting diagonal should be read together with the per-category table, where the weak rare categories are easier to spot.

10. Results and Evaluation

Audit by complaint length

We counted the alphabetic words in each original narrative and defined short as 50 words or fewer, medium as 51 to 199 words and long as 200 words or more. These are simple length measures. They are not judgements about writing quality or formality.

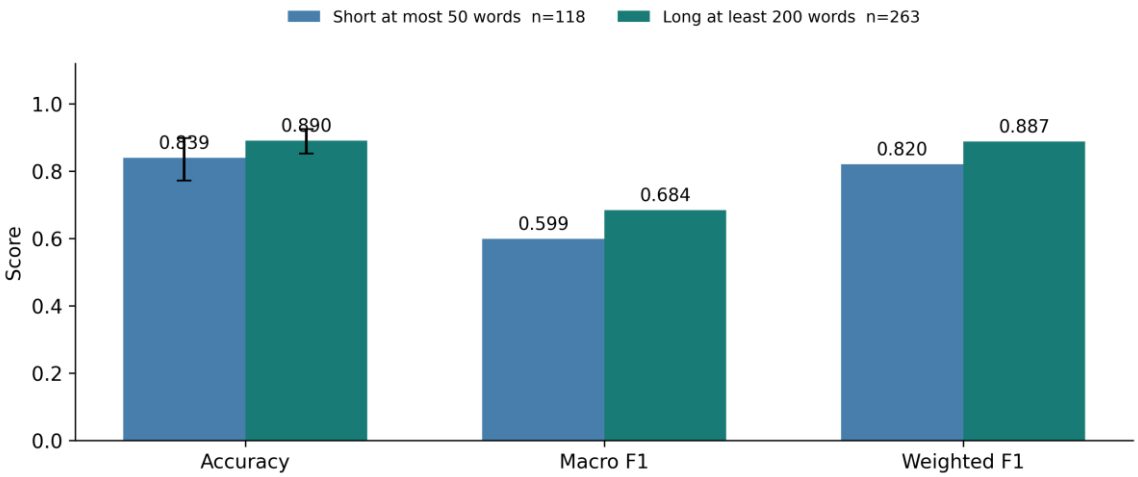


Figure 8. Short versus long complaint performance. Error bars on accuracy are the saved 95% bootstrap intervals. Subgroup Macro-F1 uses the labels present in each subgroup.

Table 11. Saved held-out results by text length

Group	n	Accuracy	Macro-F1	Weighted-F1
short	118	83.90%	0.5987	0.8196
medium	376	90.43%	0.6702	0.9006
long	263	88.97%	0.6844	0.8874

Long complaints have an accuracy of 88.97% against 83.90% for short ones, a difference of 5.08 percentage points. Medium complaints do best at 90.43%, so it is not true that more words always give better results. The category mix, product clues and the small number of rare products can all affect this comparison.

The subgroup Macro-F1 values are averaged over the labels that appear in each group: eight labels for short text and nine for long text. If we recompute short-text Macro-F1 over all nine labels, it drops to 0.5321, while long-text stays at 0.6844. Because the label sets and class mixes differ, the original Macro-F1 gap should be read with care.

10. Results and Evaluation

Audit of writing-style proxies and class mix

Our informal-short rule needs 50 words or fewer plus a cue such as “u”, “ur”, “pls”, “lol”, “gonna”, “idk”, or repeated exclamation or question marks. Our formal-detailed rule needs 200 words or more, a cue such as “furthermore”, “pursuant”, “regarding”, “documentation” or “attached”, and no informal cue. Everything else is left unassigned. These are rules we coded ourselves, and nobody has independently annotated the complaints for formality.

Table 12. Style-proxy audit from the held-out test set

Proxy group	n	Accuracy	Accuracy interval
Informal short	5	80.00%	40.00% to 100.00%
Formal detailed	113	85.84%	77.88% to 91.15%
Unassigned	639	89.51%	87.01% to 91.71%

Only five complaints meet the informal-short rule, and they cover three true categories. That is too few to compare fairly against the 113 formal-detailed complaints. So this audit does not show that the classifier is biased against informal writers. The 639 unassigned complaints also show that our cues cover only a small part of the data.

Table 13. Additional balanced short-versus-long comparison within eligible categories

Category	n per group	Short accuracy	Long accuracy
Bank accounts	7	57.14%	71.43%
Credit cards	5	80.00%	100.00%
Credit reporting	81	95.06%	95.06%
Debt collection	16	62.50%	87.50%

For each category with at least five complaints in both length groups, we sampled equal numbers using seed 42. The average of the four long-minus-short gaps is 14.82 percentage points. This controls for part of the class-mix difference, but it uses a small subset and a single sample. We see it as an exploratory observation, not a causal estimate or a full fairness check.

No category has enough paired informal-short and formal-detailed examples to run the same controlled comparison. We made a template for human annotation, but independent ratings, adjudication and agreement scores have not been done. We also did not measure demographic fairness, since the data has no reliable demographic labels.

11. Error Analysis / NLP Failure Analysis

The test set has 84 errors. We read the actual held-out narratives instead of only looking at scores. The causes we suggest below are our own interpretation of the text and the learned features. We have not confirmed them with a separate experiment.

Case 1 Missing product context

Recorded product: Debt collection. Predicted product: Credit reporting. The narrative is 15 words long and reads, “Year of 2023 I am requesting a assignment contract that which was not given to me.” It asks for a document but says almost nothing about the product, and once the year is removed there is even less to go on. The model still picks a category, which shows why a human needs to review these.

Case 2 Overlapping financial vocabulary

Recorded product: Payday/title/personal loans. Predicted product: Credit reporting. An excerpt reads, “While checking my credit report I noticed XXXX/Oportun reporting two accounts that I did not apply for.” The full narrative has 29 alphabetic words. The words credit, report and reporting strongly trigger credit-reporting features, even though the recorded label is a loan product. The full text and exact explanation are saved in error_explanation.json.

The winning credit-reporting score is -0.207662. A negative winning score is possible because every category gets its own one-versus-rest decision score, and the model simply picks the largest. It is not a negative probability. The positive contributions from “credit” and “report” explain part of the winning score, and the intercept and all other active terms make up the rest.

Case 3 Account and card ambiguity

Recorded product: Credit cards. Predicted product: Bank accounts. This 93-word complaint begins, “Capitol One locked me out of my account, and now won’t give me access to my credit”. It is mostly about access, transactions, identity documents and suspected fraud, which also come up in bank-account complaints. The model cannot reliably tell the exact product from this context.

Implications for improvement

These failures point to a few fixes: collect more unique examples for the small classes, review source labels that look ambiguous, try character n-grams to handle spelling variation, and test contextual representations. More detail in the text will not fix vocabulary overlap by itself. Explanations show what the classifier relied on, but a sensible-looking explanation can still go with a wrong category.

12. Ethical Considerations and Limitations

Unequal product performance and representation

The largest product category dominates the data. Balanced class weights reduce its influence in training, but the six payday/title/personal-loan test cases still get an F1 of zero. A wrong routing can delay a response or hide a specialist issue, so reliability should be checked product by product, especially before any automatic routing is used.

Complaint length and writing style are not protected demographic attributes. Checking them can show usability problems, but it cannot tell us whether the model is fair across age, gender, income, disability or language skill. Our style rules can also mislabel a complaint. We make no claim about demographic fairness, and we treat the formality objective as only partly evaluated.

Privacy and appropriate use

Complaint narratives can still contain sensitive information after the source redaction. Our cleaning removes obvious email patterns, numbers and redaction markers, but it is not a full personal-data detector. Names, locations and unusual events can remain. A real system would need stronger redaction, access controls, retention rules and a way for reviewers to correct or appeal a routing suggestion.

The classroom demo processes input locally and needs a human to review every suggestion. It should not be used to decide creditworthiness, whether a complaint is valid, eligibility for help or legal outcomes. We treat the recorded CFPB product label as the target for classification, not as proof that an allegation is true.

Scores and proposed triage thresholds

Raw SVM decision margins are uncalibrated, so they should not be shown as confidence percentages. The demo rejects inputs with fewer than three cleaned words or no recognised vocabulary, and it flags every prediction for human review. It does not route a complaint automatically based on a confidence threshold.

In future, a triage rule could send a complaint to a general review queue when the absolute winning score is below 0.10, or when the winning and second-highest scores are less than 0.10 apart. These thresholds are only proposals. They are not implemented or validated. Before any deployment, the false-routing and review-volume trade-offs would need to be measured on validation data, with extra attention to rare products and all-negative scores.

Scope of the evaluation

Our experiment uses one small snapshot from a short date window, a random split instead of a chronological one, nine retained products and a single validation-based model comparison. Exact duplicates are removed, but near-duplicate templates may remain. Amounts, dates, emojis and non-English scripts are discarded in preprocessing. We have not done an out-of-domain test, a multilingual benchmark, a human formality agreement study or probability calibration.

13. Conclusion

We built a pipeline that classifies financial complaints using real CFPB narratives, clearly documented data preparation, TF-IDF features and a linear SVM chosen by validation Macro-F1. On the held-out test set it reaches 88.90% accuracy, a Weighted-F1 of 0.8842 and a Macro-F1 of 0.6834. The majority baseline is much weaker, so the learned text features clearly help on this snapshot.

The audit shows that the overall score hides uneven results across products and lower accuracy on short complaints. The six payday/title/personal-loan test cases were all missed. We could not reach a conclusion on formal versus informal writing, because the informal group has only five cases and the style labels have not been independently checked.

Looking at global features and exact local contributions lets us see which words the SVM relies on. The Colab notebook, browser demo and model card let us show both good predictions and real mistakes. The project supports routing with human oversight. It does not show that the model is ready for fully automatic complaint handling.

14. Future Scope

- Collect data over a longer period and get more unique narratives for the small and excluded categories. Test on a later time period to see how much the model drifts.
- Build a balanced, human-labelled dataset for formality and level of detail, with a clear annotation rubric, independent raters and inter-rater agreement, before making any claims about writing style.
- Try character n-grams and contextual embeddings against the TF-IDF baseline, using the same split protocol and reporting per-category results.
- Look at complaints that involve more than one product and check whether a multi-label setup would suit them better than a single label.
- Calibrate the scores and validate abstention or review thresholds on validation data, reporting coverage, review volume and error rates for each product.
- Improve personal-data redaction and add reviewer feedback, monitoring and correction tools before any real operational use.

15. References

References are numbered. The final results come from the project's saved experiment files, and the external sources below describe the dataset and methods.

[1] Consumer Financial Protection Bureau. Consumer Complaint Database. Online documentation. Accessed 9 October 2026.

https://www.consumerfinance.gov/data-research/consumer-complaints/

[2] Data.gov. Consumer Financial Protection Bureau Consumer Complaint Database. Online documentation. Accessed 9 October 2026.

https://catalog.data.gov/dataset/consumer-complaint-database

[3] scikit-learn developers. TfidfVectorizer documentation. Online documentation. Accessed 9 October 2026.

https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

[4] scikit-learn developers. LinearSVC documentation. Online documentation. Accessed 9 October 2026.

https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html

[5] scikit-learn developers. LogisticRegression documentation. Online documentation. Accessed 9 October 2026.

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

[6] scikit-learn developers. RandomForestClassifier documentation. Online documentation. Accessed 9 October 2026.

https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

[7] scikit-learn developers. Metrics and scoring quantifying the quality of predictions. Online documentation. Accessed 9 October 2026.

https://scikit-learn.org/stable/modules/model_evaluation.html

[8] pandas development team. pandas documentation. Online documentation. Accessed 9 October 2026.

https://pandas.pydata.org/docs/

[9] Matplotlib development team. Matplotlib documentation. Online documentation. Accessed 9 October 2026.

https://matplotlib.org/stable/

Project evidence: Financial_Complaints_Colab.ipynb; data_manifest.json; filtering_counts.csv; validation_comparison.csv; test_metrics.json; classification_report.csv; audit.csv; controlled_audit.json; demo_prediction.json; error_explanation.json; and test_predictions.csv.

16. GitHub Repository

GitHub Repository Link

https://github.com/shoryaj/financial-complaint-classification-

The Colab notebook is the main way to reproduce our work. The project package also has the team model card, an explanation and run guide, and the verified output files. The notebook generates the fitted model, metrics, plots, audit files and prediction records. It includes the saved dataset, so a classroom run does not need to download the live database.

Table 14. Files required for a complete reproducibility submission

File or group	Purpose
Financial_Complaints_Colab.ipynb	Complete code, embedded data and executed output cells.
Project_Explanation_and_ Colab_Instructions.md	Run instructions and explanation of each stage.
Team_Model_Card.pdf and Team_Model_Card.png	One-page intended-use, results and limitation summary.
Financial_Complaint_Live_Demo.html	Standalone offline prediction and output demonstration.
Generated outputs	Saved model, source manifest, split identifiers, metrics, audit and error files.
This project report	Full project account in the SAII report structure.

17. Team Contribution

All four members worked together on every stage of the project. This includes dataset preparation, modelling, evaluation, performance auditing, explainability, documentation and preparing the demo.

Table 15. Team members and jointly reported contribution

Team Member	PRN	Specific Contribution
Shorya Jaiswal	25030422113	Joint contribution across all stages listed above.
Namir Al Balushi	25030422151	Joint contribution across all stages listed above.
Abdullah Al Hinai	25030422149	Joint contribution across all stages listed above.
Yousuf Altom	25030421050	Joint contribution across all stages listed above.

18. Annexure (if required)

Annexure A One page model card

FinancialComplaint v2 Academic prototype 9 October 2026

Model details

Nine-product classifier using TF-IDF unigrams and bigrams (30,000 features) and a balanced Linear SVM with C=1. CFPB snapshot of 10,000 records; 5,042 kept after removing seven short, 190 conflicting-label, 4,724 duplicate and 37 rare-label records; no blank fields. Stratified train/validation/test split of 3,529/756/757 with seed 42.

Intended use

English-oriented classroom demonstration and human-reviewed product-routing suggestions. Credit decisions, legal judgements, complaint validity and verified multilingual use are outside what we evaluated.

Table 16. Model card held-out performance

Model	Accuracy	Macro-F1	Weighted-F1
Majority baseline	71.33%	0.0925	0.5940
Linear SVM C=1	88.90%	0.6834	0.8842

Correct predictions: 673/757. The 95% bootstrap accuracy interval is 86.66% to 91.02%. Credit reporting has n=540 and F1=0.9460, while payday/title/personal loans has n=6 and F1=0.0000. The common category dominates the weighted result.

Subgroup audit

Short-text accuracy is 83.90% (n=118) and long-text accuracy is 88.97% (n=263), a gap of 5.08 points. Formal-detailed accuracy is 85.84% (n=113) and informal-short is 80.00% (n=5). The style comparison is inconclusive and demographic fairness was not measured. Subgroup Macro-F1 uses different label sets.

Explainability and outputs

Global coefficients and exact TF-IDF times coefficient contributions show which terms matter most. The mortgage demo score is 0.627661. Margins are uncalibrated and are not confidence percentages. Inputs with fewer than three cleaned words or no recognised vocabulary are rejected. Every suggestion needs human review.

Limitations and recommendations

The snapshot is small and imbalanced, and near-duplicates and ambiguous source labels may remain. Filtering to English characters removes other scripts and some useful numbers. Next steps: add unique rare-product examples and human-annotated style examples, test later periods and improve redaction. The general-queue rules (winning score below 0.10, or a top-two score gap below 0.10) are proposed examples only, not implemented or validated.

Development team: Shorya Jaiswal, Namir Al Balushi, Abdullah Al Hinai and Yousuf Altom, with joint work across all stages. AI assistance supported code and documentation. Evidence: saved notebook, metrics, prediction records and audit files.

18. Annexure (if required)

Annexure B Explainability analysis

Global inspection ranks the top positive SVM coefficients for each product. How much a term matters for a particular complaint also depends on its TF-IDF value. Some top terms are company names or generic words, which shows the model has picked up patterns from this corpus rather than real product definitions. These rankings are best read together with the held-out errors.

Table 17. Three highest positive coefficients per retained category

Category	Top terms and coefficient values
Bank accounts	checking (1.882); bank (1.866); chime (1.724)
Credit cards	card (3.043); credit card (2.272); interest (1.527)
Credit reporting	these (1.586); equifax (1.493); report (1.468)
Debt collection	debt (3.130); collection (1.916); owe (1.354)
Money services	paypal (1.506); cash (1.453); cash app (1.328)
Mortgages	mortgage (3.392); escrow (1.817); my mortgage (1.535)
Payday/title/personal loans	loan (1.179); affirm (1.070); the loan (1.057)
Student loans	mohela (1.915); student (1.664); loans (1.405)
Vehicle loans	car (1.939); vehicle (1.878); permission to (1.204)

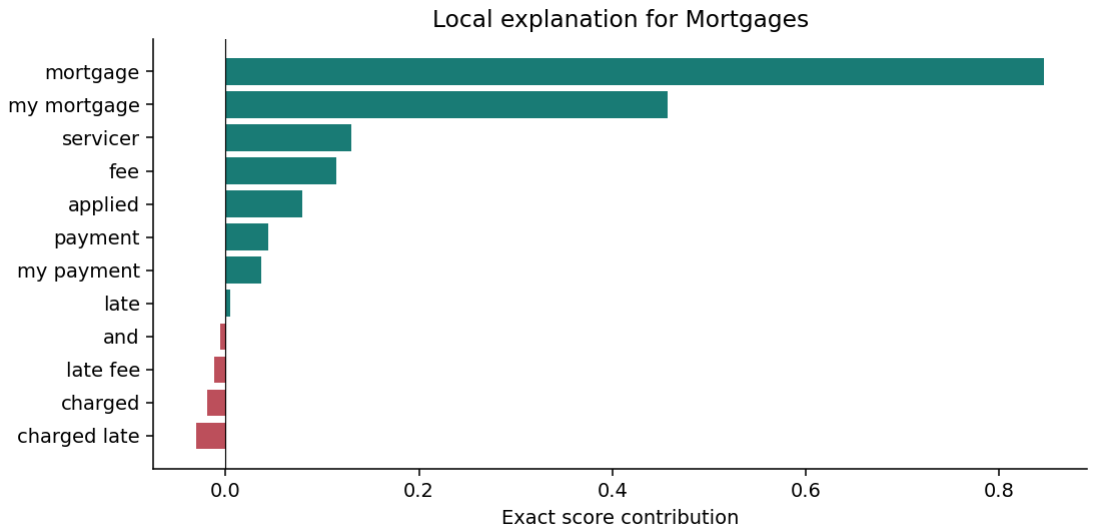


Figure 9. Exact local contributions for the mortgage demo, preserved from the executed notebook display output.

Each local contribution is the term's TF-IDF value multiplied by the coefficient for the predicted class. The score is the intercept plus all active contributions: -1.010147 plus 1.637808 gives 0.627661 for the mortgage demo. The plot shows selected terms only. This is direct linear attribution, not SHAP or LIME, and it does not show causal effects.

18. Annexure (if required)

Annexure C Reproduction and live demo steps

1. Open Financial_Complaints_Colab.ipynb in Google Colab. If an earlier package installation changed the runtime, choose Runtime > Disconnect and delete runtime, then reconnect.

2. Choose Runtime > Run all. The setup checks the installed libraries without upgrading or downgrading NumPy. Confirm that the dataset identity check passes and that 5,042 narratives are kept.

3. Read the printed split counts and validation comparison, and confirm that Linear SVM C=1 was chosen using validation data only.

4. Show the evaluation output: 673 of 757 correct, accuracy 88.90%, Macro-F1 0.6834 and Weighted-F1 0.8842. Then show the category bars, the confusion matrix and the subgroup audit.

5. Enter a complaint in the notebook widget or the direct prediction cell, and explain the predicted category and the contribution plot. Then show a recorded misclassification next to its source label.

6. For the website demo, open the hosted site with the permitted account, or double-click Financial_Complaint_Live_Demo.html in a browser. Classify a complaint, open Performance and download the model card.

7. Download the generated Colab output ZIP and keep its model, metrics, split identifiers, audit files and raw prediction records with the submission.

Source identity

The preserved source CSV contains 10,000 records. Its SHA-256 digest is:

```
3bd2a4ef07eff6c665e3b6cbc20f91600874b5ae629f39bdaf9ae407b98a4794
```

The digest identifies the exact snapshot used in our experiment. A newer CFPB download may contain different records, so its filtering counts and metrics may not match unless the evaluation is run again.

Expected saved outputs

The notebook saves model.joblib, data_manifest.json, the train/validation/test identifier CSV files, test_predictions.csv, test_metrics.json, baseline_metrics.json, validation_comparison.csv, classification_report.csv, audit.csv, controlled_audit.json, top_features.csv, error_examples.csv and the explanation JSON files. It also saves the performance, confusion, subgroup and local-attribution figures. The human-audit template is blank and does not represent completed human annotation.